

LAB 1

Objective: To solve non-linear equation using bisection method

Theory

The **bisection method** is a numerical technique used to find the root of a continuous function $f(x)=0$ within a given interval $[a,b]$. The method works on the principle of repeatedly bisecting the interval and selecting the subinterval where the root lies.

Algorithm

Step 1: Start

Step 2: Define the function $f(x)$.

Step 3: Choose initial guesses a and b such that $f(a) \cdot f(b) < 0$.

Step 4: Set the pre-specified tolerance e .

Step 5: Compute midpoint $c = (a+b)/2$.

Step 6: Evaluate $f(c)$:

- If $f(a) \cdot f(c) < 0$, set $b = c$
- If $f(b) \cdot f(c) < 0$, set $a = c$
- If $f(c) = 0$, go to Step 8.

Step 7: If $|b-a| > e$, repeat Step 5; otherwise, go to Step 8.

Step 8: Display c as the root.

Step 9: Stop.

Program

```
//x^2 - 27 = 0
#include<stdio.h>
#include<math.h>
float f(float);
float g(float);
int main()
{
    float a = 5, b = 6;
    float tol = 1E-4;
    float m;
    while(fabs(b - a)/2 > tol)
    {
        m = (a + b)/2;
        if(f(m) * f(a) < 0)
        {
            b = m;
        }
        else
        {
            a = m;
        }
    }
    printf("%f\n", m);
}
```

```
    }  
    return 0;  
}  
float f(float x)  
{  
    return x * x - 27;  
}
```

Output

```
5.500000  
5.250000  
5.125000  
5.187500  
5.218750  
5.203125  
5.195312  
5.199219  
5.197266  
5.196289  
5.195801  
5.196045  
5.196167  
  
=== Code Execution Successful ===|
```

Conclusion

The bisection method is a simple and reliable numerical technique for solving non-linear equations. By iteratively halving the interval and evaluating the function, it efficiently converges to the root with a guaranteed error bound.

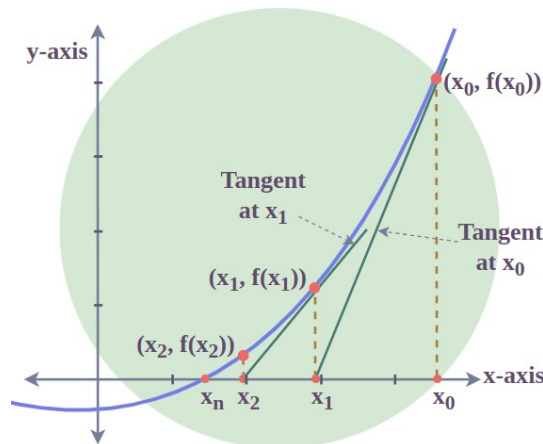
LAB 2

Objective: To find the root of non-linear equation using Newton-Raphson method in c

Theory

The Newton-Raphson method is a powerful numerical technique used to find the approximate roots of a real-valued function $f(x)$. It is an iterative algorithm that converges quickly under certain conditions. This method is particularly useful when the derivative of the function is easily computable. The Newton Raphson Method is referred to as one of the most commonly used techniques for finding the roots of given equations. It can be efficiently generalized to find solutions to a system of equations.

The geometric meaning of Newton's Raphson method is that a tangent is drawn at the point $(x_0, f(x_0))$ to the curve $y = f(x)$.



It cuts the x-axis at x_1 , which will be a better approximation of the root. Now, drawing another tangent at $(x_1, f(x_1))$, which cuts the x-axis at x_2 , which is a still better approximation and the process can be continued till the desired accuracy is achieved.

Algorithm

Step 1: Start

Step 2: Define function as $f(x)$

Step 3: Define first derivative of $f(x)$ as $g(x)$

Step 4: Input initial guess (x_0) , tolerable error (e) and maximum iteration (N)

Step 5: Initialize iteration counter $i = 1$

Step 6: If $g(x_0) = 0$ then print "Mathematical Error" and goto (12) otherwise goto (7)

Step 7: Calculate $x_1 = x_0 - f(x_0) / g(x_0)$

Step 8: Increment iteration counter $i = i + 1$

Step 9: If $i \geq N$ then print "Not Convergent" and goto (12) otherwise goto (10)

Step 10: If $|f(x_1)| > e$ then set $x_0 = x_1$ and goto (6) otherwise goto (11)

Step 11: Print root as x_1

Step 12: Stop

Program

```
#include<stdio.h>
#include<math.h>

float f(float);
float df(float);

int main()
{
    float x = 2;
    float tol = 1E-5;
    while(fabs(f(x)) > tol)
    {
        x = x - f(x)/df(x);
        printf("%f\n", x);
    }
    return 0;
}

float f(float x)
{
    return pow(x, 2) - 27;
}

float df(float x)
{
    return 2 * x;
}
```

Output

```
Output
7.750000
5.616936
5.211914
5.196176
5.196152

=== Code Execution Successful ===
```

Conclusion

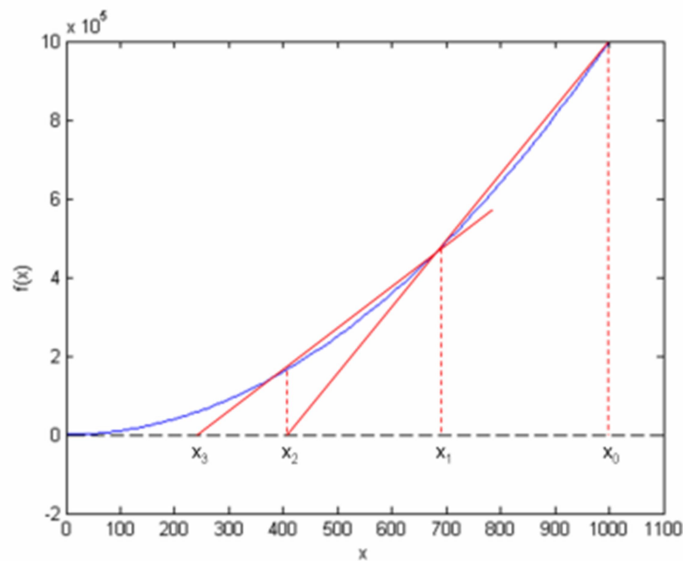
This code uses the Newton-Raphson method to find the root of the nonlinear equation $f(x)=x^2-27$. At each iteration, the current approximation of the root is printed. The process converges quickly because the Newton-Raphson method has quadratic convergence when the initial guess is reasonably close to the actual root.

Lab 3

Objective: To find the root of non-linear equation using secant method in c.

Theory:

The Secant Method is a numerical technique for finding the approximate root of a real-valued function $f(x)=0$. Unlike the Newton-Raphson method, which uses the derivative of the function, the secant method relies only on function values, making it derivative-free. It is a recursive method for finding the root of polynomials by successive approximation.



Formula for the Secant Method is

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Algorithm

Step 1: Start

Step 2: Define function as $f(x)$

Step 3: Input initial guesses (x_0 and x_1), tolerable error (e) and maximum iteration (N)

Step 4: Initialize iteration counter $i = 1$

Step 5: If $f(x_0) = f(x_1)$ then print "Mathematical Error" and goto (11) otherwise goto (6)

Step 6: Calculate $x_2 = x_1 - (x_1 - x_0) * f(x_1) / (f(x_1) - f(x_0))$

Step 7: Increment iteration counter $i = i + 1$

```
#include <stdio.h>

#include <math.h>

double f(double x) {
    return sin(x) - 2 * x + 1; // f(x) = sin(x) - 2x + 1
}

// Secant method function

void secant(double x0, double x1, double tol, int max_iter) {
    double x2, f0, f1, f2;

    int iter = 0;

    printf("Iter\t\ttx0\t\t\ttx1\t\t\ttx2\t\t\ttf(x2)\n");

    printf("-----\n");

    do {
        f0 = f(x0);

        f1 = f(x1);

        // Prevent division by zero

        if (fabs(f1 - f0) < 1e-10) {

            printf("Error: Division by zero encountered in the iteration.\n");

            return;

        }

        // Secant method formula

        x2 = x1 - f1 * (x1 - x0) / (f1 - f0);

        f2 = f(x2);

        printf("%d\t\t%.6f\t\t%.6f\t\t%.6f\t\t%.6f\n", iter + 1, x0, x1, x2, f2);

        // Update values for next iteration

        x0 = x1;

        x1 = x2;

        iter++;

        if (fabs(f2) < tol || iter >= max_iter) {
```

```

    break;

}

} while (fabs(x1 - x0) >= tol);

if (fabs(f2) < tol) {
    printf("\nRoot found: %.6f (after %d iterations)\n", x2, iter);
} else {
    printf("\nMethod did not converge within the maximum number of
iterations.\n");
}
}

int main() {
    double x0, x1, tol;

    int max_iter;

    printf("Enter the first initial guess (x0): ");
    scanf("%lf", &x0);

    printf("Enter the second initial guess (x1): ");
    scanf("%lf", &x1);

    printf("Enter the tolerance: ");
    scanf("%lf", &tol);

    printf("Enter the maximum number of iterations: ");
    scanf("%d", &max_iter);

    // Call the secant method function
    printf("\nFinding the root using the Secant Method...\n");

    secant(x0, x1, tol, max_iter);

    return 0;
}

```

Output

```
Output
Enter the first initial guess (x0): 1
Enter the second initial guess (x1): 2
Enter the tolerance: 3
Enter the maximum number of iterations: 4

Finding the root using the Secant Method...
Iter      x0      x1      x2      f(x2)
-----
1         1.000000   2.000000   0.917953  -0.041546

Root found: 0.917953 (after 1 iterations)

=== Code Execution Successful ===
```

Conclusion

The provided program employs the Secant Method to approximate the root of the nonlinear equation $f(x) = \sin(x) - 2x + 1$. This iterative method uses two initial guesses i.e. x_0 and x_1 , to compute successive approximations of the root without requiring the derivative of the function. The program efficiently finds the root and demonstrates the Secant Method's utility in solving equations where derivatives are unavailable or complex.

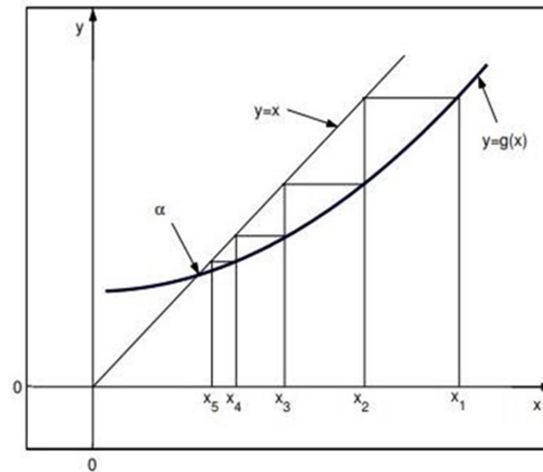
Lab 4

Objective: To find the root of non-linear equation using fixed-point iteration method in c.

Theory

The fixed-point iteration method in numerical analysis is used to find an approximate solution to algebraic and transcendental equations. Sometimes, it becomes very tedious to find solutions to cubic, bi-quadratic and transcendental equations; then, we can apply specific numerical methods to find the solution; one among those methods is the fixed-point iteration method.

The fixed-point iteration method uses the concept of a fixed point in a repeated manner to compute the solution of the given equation. A fixed point is a point in the domain of a function g such that $g(x) = x$. In the fixed-point iteration method, the given function is algebraically converted in the form of $g(x) = x$.



Algorithm

Step 1: Start

Step 2: Define function $f(x)$

Step 3: Define function $g(x)$ which is obtained from $f(x)=0$ such that $x = g(x)$ and $|g'(x)| < 1$

Step 4: Choose initial guess x_0 , Tolerable Error e and Maximum Iteration N

Step 5: Initialize iteration counter: step = 1

Step 6: Calculate $x_1 = g(x_0)$

Step 7: Increment iteration counter: step = step + 1

Step 8: If step > N then print "Not Convergent" and goto (12) otherwise goto (10)

Step 9: Set $x_0 = x_1$ for next iteration

Step 10: If $|f(x_1)| > e$ then goto step (6) otherwise goto step (11)

Step 11: Display x_1 as root.

Step 12: Stop

Program

```
#include <stdio.h>
#include <math.h>

// Define the g(x) function for fixed-point iteration
double g(double x) {
    return cbrt((5 + 2 * x) / 2); // g(x) = cube root of ((5 + 2x) / 2)
}

int main() {
    double x0, tol;
    int max_iter, iter = 0;
    double x1;

    // Input initial guess, tolerance, and maximum iterations
    printf("Enter the initial guess (x0): ");
    scanf("%lf", &x0);
    printf("Enter the tolerance: ");
    scanf("%lf", &tol);
    printf("Enter the maximum number of iterations: ");
    scanf("%d", &max_iter);
    printf("\nIter\t\tx0\t\t\tx1\t\t\tError\n");
    printf("-----\n");

    // Fixed-Point Iteration
    do {
        x1 = g(x0); // Compute the next approximation
        printf("%d\t\t%.6f\t\t%.6f\t\t%.6f\n", iter + 1, x0, x1, fabs(x1 - x0));

        // Check for convergence
        if (fabs(x1 - x0) < tol) {
```

```

    printf("\nThe root is approximately: %.6f (after %d iterations)\n",
x1, iter + 1);

    return 0;
}

x0 = x1; // Update x0 for the next iteration

iter++;

} while (iter < max_iter);

// If maximum iterations are reached without convergence

printf("\nThe method did not converge within the maximum number of
iterations.\n");

return 1;
}

```

Output

Output

Enter the initial guess (x0): 0
Enter the tolerance: 1
Enter the maximum number of iterations: 10

Iter	x0	x1	Error
1	0.000000	1.357209	1.357209
2	1.357209	1.568283	0.211074

The root is approximately: 1.568283 (after 2 iterations)

=== Code Execution Successful ===

Conclusion

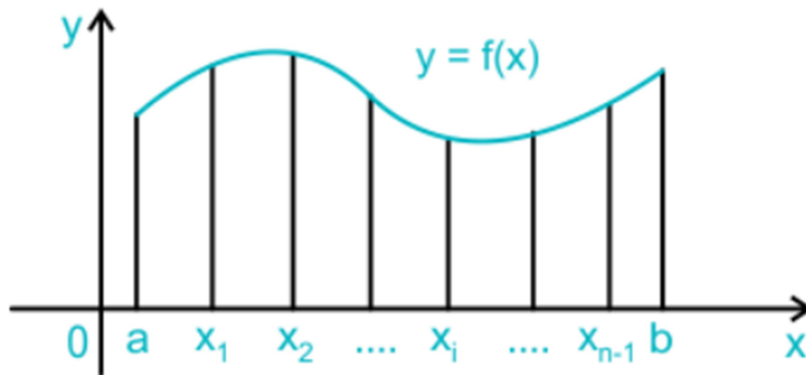
The fixed-Point Iteration function implements the Fixed-Point Iteration method. It iterates until the specified maximum number of iterations or until the solution converges within the given tolerance. In the main function, you can set the initial guess, tolerance, and maximum number of iterations according to your problem. The fixed-Point Iteration function is called to find the root, and the result is printed.

Lab 5

Objective: To find the root of non-linear equation using trapezoidal method in c.

Theory

The Trapezoidal Method is a numerical technique used to approximate the definite integral of a function over a given interval. It is based on approximating the region under the curve by dividing it into smaller subintervals and representing each subinterval by a trapezoid. The total area of these trapezoids is then used to estimate the integral.



The accuracy of the trapezoidal rule depends on the number of subintervals (n). As you increase the number of subintervals, the approximation tends to get closer to the actual integral value. Mathematically, the error decreases as n increases, and it converges to the true value as n approaches infinity.

Algorithm

Step 1: Start

Step 2: Define function $f(x)$

Step 3: Read lower limit of integration, upper limit of integration and number of sub
Interval.

Step 4: Calculate: step size = (upper limit - lower limit)/number of sub interval

Step 5: Set: integration value = $f(\text{lower limit}) + f(\text{upper limit})$

Step 6: Set: $i = 1$

Step 7: If $i > \text{number of sub interval}$ then goto

Step 8: Calculate: $k = \text{lower limit} + i * h$

Step 9: Calculate: $\text{Integration value} = \text{Integration Value} + 2 * f(k)$

Step 10: Increment i by 1 i.e. $i = i+1$ and go to step 7

Step 11: Calculate: $\text{Integration value} = \text{Integration value} * \text{step size}/2$

Step 12: Display Integration value as required answer

Step 13: Stop

Program

```
#include <stdio.h>

// Function to calculate f(x)
double f(double x) {
    return x * x + 1; // Example: f(x) = x^2 + 1
}

// Trapezoidal Method function
double trapezoidal(double a, double b, int n) {
    double h = (b - a) / n; // Width of each subinterval
    double sum = f(a) + f(b); // Sum of first and last terms
    // Calculate the sum of intermediate terms
    for (int i = 1; i < n; i++) {
        double x = a + i * h;
        sum += 2 * f(x); // Multiply by 2 for intermediate terms
    }
    // Multiply by h/2 to get the final integral value
    return (h / 2) * sum;
}

int main() {
    double a, b;
    int n;

    // Input the interval and number of subintervals
    printf("Enter the lower limit of integration (a): ");
    scanf("%lf", &a);
    printf("Enter the upper limit of integration (b): ");
    scanf("%lf", &b);
    printf("Enter the number of subintervals (n): ");
```

```
scanf("%d", &n);

// Ensure n is positive
if (n <= 0) {
printf("Error: Number of subintervals must be positive.\n");
return 1;
}

// Calculate the integral using the Trapezoidal Method
double result = trapezoidal(a, b, n);

// Display the result
printf("\nThe approximate value of the integral is: %.6f\n", result);

return 0;
}
```

Output

```
Output
Enter the lower limit of integration (a): 0
Enter the upper limit of integration (b): 1
Enter the number of subintervals (n): 10

The approximate value of the integral is: 1.335000

=== Code Execution Successful ===
```

Conclusion

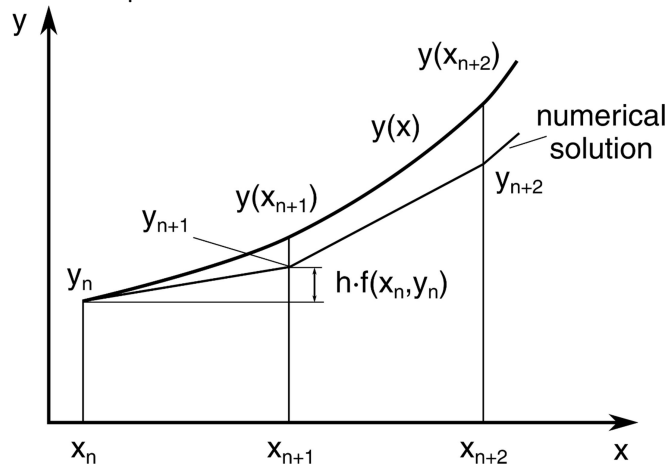
The trapezoidal method is a numerical approximation technique and is particularly useful when you need to compute the integral of a function for which there is no known closedform expression or when the integral is difficult to evaluate analytically. It's relatively easy to implement and is one of the simplest numerical integration methods.

Lab 6

Objective: To find the root of a non-linear polynomial using Euler's method

Theory

Euler's method is a basic numerical technique for approximating solutions to ordinary differential equations (ODEs). It's a first-order method, meaning that the error in the approximation is proportional to the size of the step used in the method. Here's how Euler's method works:



Algorithm

Step 1: Start

Step 2: Define function $f(x,y)$

Step 3: Read values of initial condition (x_0 and y_0), number of steps (n) and calculation point (x_n)

Step 4: Calculate step size (h) = $(x_n - x_0)/n$

Step 5: Set $i=0$

Step 6: Loop

$y_n = y_0 + h * f(x_0 + i*h, y_0)$ $y_0 = y_n$

$i = i + 1$ While $i < n$

Step 7: Display y_n as result

Step 8: Stop

Program

```

#include <stdio.h>

#include <math.h> // Include the math library

// Define the function representing the ODE  $dy/dx = f(x, y)$ 
double f(double x, double y) {
    // Example:  $dy/dx = x + y$ 
    return x + y;
}

// Euler's method function
void euler(double x0, double y0, double h, double xn) {
    double x = x0;
    double y = y0;
    while (x < xn) {
        printf("x = %lf, y = %lf\n", x, y);
        // Update y using Euler's method
        y += h * f(x, y);
        // Update x
        x += h;
    }
}

int main() {
    // Initial conditions
    double x0 = 0.0;
    double y0 = 1.0; // Initial value of y at x0
    double h = 0.1; // Step size
    double xn = 2.0; // Desired x value for the solution

    printf("Approximate solution using Euler's method:\n");
    euler(x0, y0, h, xn);
    return 0;
}

```

```
}
```

Output

Output

Approximate solution using Euler's method:

```
x = 0.000000, y = 1.000000
x = 0.100000, y = 1.100000
x = 0.200000, y = 1.220000
x = 0.300000, y = 1.362000
x = 0.400000, y = 1.528200
x = 0.500000, y = 1.721020
x = 0.600000, y = 1.943122
x = 0.700000, y = 2.197434
x = 0.800000, y = 2.487178
x = 0.900000, y = 2.815895
x = 1.000000, y = 3.187485
x = 1.100000, y = 3.606233
x = 1.200000, y = 4.076857
x = 1.300000, y = 4.604542
x = 1.400000, y = 5.194997
x = 1.500000, y = 5.854496
x = 1.600000, y = 6.589946
x = 1.700000, y = 7.408941
x = 1.800000, y = 8.319835
x = 1.900000, y = 9.331818
```

```
=== Code Execution Successful ===|
```

Conclusion

Euler's method is a relatively simple and less accurate numerical method for solving ODEs. Other more sophisticated methods like the Runge-Kutta methods or adaptive step-size methods are often used for higher accuracy and stability.

Lab 7

Objective: To find integration using Simpson's 1/3 rule.

Theory

Simpson's 1/3 Rule is a numerical method used for approximating the definite integral of a function. It provides a more accurate approximation compared to the Trapezoidal Rule by using parabolic interpolation. The method is based on approximating the function by a second-degree polynomial and integrating that polynomial over the given interval.

Mathematically, the integral of a function over an interval is given by:

$$\int_a^b f(x)dx \approx \frac{h}{3} \left[f(a) + 4 \sum_{i=1,3,5,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{n-2} f(x_i) + f(b) \right]$$

The accuracy of the Simpson's 1/3 Rule depends on the number of subintervals n . A higher n leads to a better approximation.

Algorithm

1. Start
2. Define function $f(x)$.
3. Read lower limit of integration, upper limit of integration, and number of subintervals n
4. Ensure that n is even. If not, display an error message and stop execution.
5. Calculate step size: $h=(b-a)/n$.
6. Compute integral value using Simpson's 1/3 formula:
 - Initialize $sum=f(a)+f(b)$.
 - Sum odd indexed terms multiplied by 4
 - Sum even indexed terms multiplied by 2
 - Multiply the total sum by $h/3$.
7. Display the computed integral value
8. Stop

Program

```
#include <stdio.h>

double f(double x) {
    return x * x + 1;
}

double simpsons_rule(double a, double b, int n) {
    if (n % 2 != 0) {
        printf("Error: Number of subintervals (n) must be even.\n");
        return -1;
    }

    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; i++) {
        double x = a + i * h;

        if (i % 2 == 0)
            sum += 2 * f(x);
        else
            sum += 4 * f(x);
    }

    return (h / 3) * sum;
}

int main() {
    double a, b;

    int n;

    printf("Enter the lower limit of integration (a): ");
    scanf("%lf", &a);

    printf("Enter the upper limit of integration (b): ");
    scanf("%lf", &b);

    printf("Enter the number of subintervals (n, must be even): ");
    scanf("%d", &n);
```

```
double result = simpsons_rule(a, b, n);

if (result != -1)

    printf("\nThe approximate value of the integral is: %.6f\n", result);

return 0;

}
```

Output

```
Enter the lower limit of integration (a): 1
Enter the upper limit of integration (b): 2
Enter the number of subintervals (n, must be even): 4

The approximate value of the integral is: 3.333333
```

Conclusion

Simpson's 1/3 Rule is a powerful numerical integration technique that provides higher accuracy compared to the Trapezoidal Rule. It uses quadratic interpolation to approximate the integral and requires the number of subintervals to be even. This method is particularly useful for computing integrals of complex functions where analytical solutions are difficult or impossible to obtain.

Lab 8

Objective: To find the root of nonlinear equation using false position method

Theory

The False Position Method (also known as the Regula Falsi Method) is a numerical technique used to find the root of a nonlinear equation $f(x) = 0$. It is based on the intermediate value theorem, which states that if a function changes sign over an interval $[a, b]$, then there is at least one root in that interval. The method improves upon the Bisection Method by using a secant line to approximate the root. The formula for updating the root estimate x_r is:

$$x_r = b - \frac{f(b)(b-a)}{f(b)-f(a)}$$

The method continues iteratively by updating either a or b based on the sign of $f(x_r)$ until the desired accuracy is achieved.

Algorithm

1. Start
2. Define function $f(x)$
3. Read initial guesses a and b .
4. Check if $f(a)$ and $f(b)$ have opposite signs. If not, display an error message and stop execution.
5. Compute x_r using the False Position formula.
6. Check for convergence using a tolerance value
7. If $f(x_r)$ is close to zero or desired accuracy is reached, stop
8. Update a or b based on the sign of $f(x_r)$.
9. Repeat steps 5–8 until convergence
10. Display the computed root
11. Stop

Program

```
#include <stdio.h>

#include <math.h>

double f(double x) {
    return x * x - 4;
}
```

```

}

double false_position(double a, double b, double tol) {
    if (f(a) * f(b) >= 0) {
        printf("Error: f(a) and f(b) must have opposite signs.\n");
        return -1;
    }

    double x_r;
    int iter = 0;
    while (1) {
        x_r = b - (f(b) * (b - a)) / (f(b) - f(a));

        if (fabs(f(x_r)) < tol) {
            break;
        }

        iter++;

        if (f(x_r) * f(a) < 0)
            b = x_r;
        else
            a = x_r;
    }

    printf("Root found at x = %.6f after %d iterations\n", x_r, iter);
    return x_r;
}

int main() {
    double a, b, tol;

    printf("Enter the first guess (a): ");
    scanf("%lf", &a);

    printf("Enter the second guess (b): ");
    scanf("%lf", &b);

    printf("Enter the tolerance: ");
    scanf("%lf", &tol);

    false_position(a, b, tol);

    return 0;
}

```

Output

```
Enter the first guess (a): 0
Enter the second guess (b): 5
Enter the tolerance: 0.0001

Root found at x = 2.000000 after 5 iterations
```

Conclusion

The False Position Method is an efficient root-finding technique that combines the advantages of the Bisection Method and Secant Method. It converges faster than the Bisection Method while ensuring the root remains bracketed within the interval $[a,b]$. This method is particularly useful when a good initial guess can be made, ensuring stable convergence to the root.

Lab 9

Objective: To find the root of non-linear equation using using secant method

Theory

The Secant Method is an iterative numerical technique used to approximate the root of a nonlinear equation $f(x)=0$. Unlike the False Position Method, which ensures bracketing, the Secant Method does not necessarily maintain a root bracket, making it more efficient but sometimes less stable.

The formula used to compute the next approximation of the root is:

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

Algorithm

1. Start
2. Define function $f(x)$
3. Read initial guesses x_0 and x_1 .
4. Set a tolerance value for convergence
5. Compute x_{n+1} using the Secant formula
6. Check for convergence using the tolerance
7. If convergence criteria are met, stop
8. Update values: $x_{n-1} = x_n$, $x_n = x_{n+1}$.
9. Repeat steps 5–8 until the root is found
10. Display the computed root
11. Stop

Program

```
#include <stdio.h>

#include <math.h>

double f(double x) {
    return x * x - 4;
}

double secant_method(double x0, double x1, double tol) {
    double x2;
```

```

int iter = 0;

while (fabs(f(x1)) > tol) {
    x2 = x1 - (f(x1) * (x1 - x0)) / (f(x1) - f(x0));
    x0 = x1;
    x1 = x2;
    iter++;
}

printf("Root found at x = %.6f after %d iterations\n", x1, iter);

return x1;
}

int main() {
    double x0, x1, tol;

    printf("Enter the first guess (x0): ");
    scanf("%lf", &x0);

    printf("Enter the second guess (x1): ");
    scanf("%lf", &x1);

    printf("Enter the tolerance: ");
    scanf("%lf", &tol);

    secant_method(x0, x1, tol);

    return 0;
}

```

Output

```

Enter the first guess (x0): 0
Enter the second guess (x1): 5
Enter the tolerance: 0.0001

Root found at x = 2.000000 after 4 iterations

```

Conclusion

The Secant Method is an efficient root-finding technique that does not require bracketing like the False Position or Bisection Methods. It generally converges faster than the Bisection Method but may fail if the function has certain behaviors (e.g., multiple roots in the range). By carefully choosing initial guesses, the Secant Method can provide an accurate root approximation efficiently.

Lab 10

Objective: To find root of nonlinear equation using Gauss elimination method

Theory

The Gauss Elimination Method is a numerical technique used to solve a system of linear equations. It transforms the system into an upper triangular form through a series of row operations, and then back-substitutes to find the values of the unknowns. This method is widely used for solving simultaneous linear equations in various engineering and scientific applications.

The general steps involved in the Gauss Elimination Method are:

1. Convert the system of equations into an augmented matrix.
2. Use forward elimination to transform the matrix into an upper triangular form.
3. Apply back-substitution to find the values of the variables.

Algorithm

1. Start
2. Read the number of equations (n)
3. Read the augmented matrix
4. Perform forward elimination to convert the matrix into an upper triangular form
5. Perform back-substitution to solve for the unknowns
6. Display the results
7. Stop

Program

```
#include <stdio.h>

void gauss_elimination(int n, double a[n][n+1], double x[n]) {
    for (int i = 0; i < n; i++) {
        for (int k = i + 1; k < n; k++) {
            double factor = a[k][i] / a[i][i];
            for (int j = i; j <= n; j++) {
                a[k][j] -= factor * a[i][j];
            }
        }
    }
}
```

```

    }

    for (int i = n - 1; i >= 0; i--) {
        x[i] = a[i][n];
        for (int j = i + 1; j < n; j++) {
            x[i] -= a[i][j] * x[j];
        }
        x[i] /= a[i][i];
    }
}

int main() {
    int n;

    printf("Enter the number of equations: ");
    scanf("%d", &n);

    double a[n][n+1], x[n];

    printf("Enter the augmented matrix (coefficients and constants):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= n; j++) {
            scanf("%lf", &a[i][j]);
        }
    }

    gauss_elimination(n, a, x);

    printf("Solution:\n");
    for (int i = 0; i < n; i++) {
        printf("x[%d] = %.6f\n", i, x[i]);
    }

    return 0;
}

```

Output

```
Enter the number of equations: 3
Enter the augmented matrix (coefficients and constants):
2 1 -1 8
-3 -1 2 -11
-2 1 2 -3

Solution:
x[0] = 2.000000
x[1] = 3.000000
x[2] = -1.000000
```

Conclusion

The Gauss Elimination Method is an efficient and systematic approach to solving systems of linear equations. It is particularly useful when dealing with large systems, as it reduces the complexity compared to direct substitution methods. However, issues like numerical instability and division by small pivot elements should be handled carefully for accurate results.